

Aproveitamento de Conhecimentos – Testes de Software

Módulo: Empréstimo e devolução de livros com multa por atraso

Aluno: Matheus Vitor — Bacharelado em Ciências da Computação

Prof. Gerson Peres — Entrega: 08/10/2026

1. Cenário e regras de negócio

Módulo de um sistema fictício de biblioteca: o usuário pega livros emprestados, devolve e paga multa se atrasar. Regras definidas:

ID	Regra
RN1	O prazo de empréstimo é de 7 dias corridos.
RN2	Cada dia de atraso gera multa de R\$ 2,00.
RN3	A multa tem teto de R\$ 50,00 por devolução.
RN4	Cada usuário pode ter no máximo 3 livros emprestados ao mesmo tempo.
RN5	Usuário com multa pendente não pode pegar novo livro (até pagar).
RN6	Livro sem exemplar disponível não pode ser emprestado.

2. Parte 1 - Fundamentação e planejamento

2.1 Erro, defeito e falha; verificação e validação

- **Erro:** engano humano. O desenvolvedor escreve `>` no lugar de `>=` ao comparar o limite de livros do usuário.
- **Defeito (bug):** o resultado do erro dentro do código, ou seja, a linha `if emprestimos > LIMITE_LIVROS`.
- **Falha:** o comportamento errado visto na execução. O sistema empresta o 4º livro ao mesmo usuário, violando a RN4.

Um erro gera um defeito, e o defeito só vira falha quando aquele trecho é executado com os dados certos.

- **Verificação** (“estamos construindo o sistema *corretamente*?”): conferir se o código segue as regras RN1 a RN6, por meio de revisão e testes automatizados. Exemplo: o teste que confere que 3 dias de atraso resultam em R\$ 6,00.
- **Validação** (“estamos construindo o sistema *certo*?”): conferir se o sistema atende a necessidade real. Exemplo: o bibliotecário usa o sistema e confirma que R\$ 2,00 por dia e teto de R\$ 50,00 é a política que a biblioteca quer.

2.2 Plano de teste resumido

Item	Descrição
Escopo	Cálculo de multa, empréstimo, devolução, pagamento de multa e notificação ao usuário.
Itens a testar	calcular_multa, Biblioteca.emprestar, devolver, pagar_multa, empréstimos_ativos e a chamada ao notificador.
Fora do escopo	Interface gráfica, banco de dados, cadastro de livros e usuários, envio real de e-mail/SMS, renovação e reserva de livros.
Tipos e níveis	Unitário e integração automatizados (pytest); regressão a cada alteração; sistema e aceitação de forma manual/simulada (ver 2.3).
Critério de entrada	Regras RN1-RN6 definidas; ambiente Python instalado; código compilando/importando sem erro.
Critério de saída	100% dos testes passando; cobertura de linhas e branches $\geq 90\%$; nenhum defeito crítico aberto.
Riscos	Regras ambíguas (ex.: dia do vencimento conta como atraso?); erro de limite (off-by-one) nas datas; notificador real indisponível; módulo só em memória.
Ambiente	Linux, Python 3.14, pytest 9, pytest-cov, git.
Papéis	Aluno: analista de requisitos, desenvolvedor, testador e responsável pelos relatórios de defeito. Professor: avaliador/cliente.

2.3 Níveis e tipos de teste do módulo

Nível	O que testaria	Justificativa
Unitário	calcular_multa com várias datas; regras isoladas de emprestar.	Funções pequenas e puras, rápidas de testar, concentram as regras RN1-RN6.
Integração	Biblioteca junto com o notificador (mock).	Garante que a devolução com multa realmente avisa o usuário.
Sistema	Fluxo completo: emprestar → atrasar → devolver → multa → pagar → emprestar de novo.	Valida as regras trabalhando juntas.
Aceitação	Bibliotecário confirma os valores e as mensagens.	Valida que o sistema serve ao negócio.

Tipo	Aplicação
Funcional	Todas as regras RN1-RN6 (principal foco deste trabalho).
Regressão	Rodar o pytest completo a cada mudança, para garantir que corrigir uma regra não quebra outra.
Desempenho	Muitos empréstimos simultâneos no horário de pico (ex.: início do semestre).
Usabilidade	Tela do balcão: mensagem clara quando o empréstimo é negado e o motivo.

Tipo	Aplicação
Segurança	Só funcionário autorizado pode registrar devolução e baixar multa (evita perdão indevido de multa).

3. Parte 2 - Projeto de casos de teste

3.1 Particionamento de equivalência e valor limite

Campo 1 - dias de atraso na devolução (RN1, RN2, RN3). Os “dias após o empréstimo” são contados a partir da data do empréstimo, e o atraso é esse valor menos 7.

Partição	Atraso (dias)	Multa esperada
P1 - no prazo	≤ 0	R\$ 0,00
P2 - atraso proporcional	1 a 24	$2 \times \text{atraso}$
P3 - teto atingido	≥ 25	R\$ 50,00

Valores limite escolhidos (dias após o empréstimo $\rightarrow$ atraso $\rightarrow$ multa): **6 $\rightarrow$ -1 $\rightarrow$ 0; 7 $\rightarrow$ 0 $\rightarrow$ 0; 8 $\rightarrow$ 1 $\rightarrow$ 2; 31 $\rightarrow$ 24 $\rightarrow$ 48; 32 $\rightarrow$ 25 $\rightarrow$ 50; 33 $\rightarrow$ 26 $\rightarrow$ 50.** Foram escolhidos porque erros de “um a mais/um a menos” costumam aparecer nas fronteiras entre as partições (último dia sem multa, primeiro dia com multa, e o dia em que o teto é atingido).

Campo 2 - quantidade de livros emprestados ao usuário (RN4).

Partição	Livros ativos	Resultado
P1 - dentro do limite	0 a 2	Empréstimo permitido
P2 - limite atingido	3	Empréstimo negado

Valores limite: **2** (permite o 3º livro) e **3** (nega o 4º livro).

3.2 Tabela de decisão (concessão do empréstimo)

Condições: C1 = usuário sem multa pendente; C2 = usuário com menos de 3 livros; C3 = livro com exemplar disponível.

Regra	C1	C2	C3	Ação
R1	S	S	S	Emprestar
R2	S	S	N	Negar (sem exemplar)
R3	S	N	S	Negar (limite de 3 livros)
R4	S	N	N	Negar
R5	N	S	S	Negar (multa pendente)
R6	N	S	N	Negar
R7	N	N	S	Negar
R8	N	N	N	Negar

Apenas R1 permite o empréstimo. R2, R3 e R5 estão cobertas por testes automatizados. As demais combinações negam por mais de um motivo.

3.3 Casos de teste formais

ID	Pré-condição	Passos	Dados de entrada	Resultado esperado	Prior.
CT01	Livro emprestado em 01/01	Devolver no dia 7	Devolução 08/01	Multa R\$ 0,00	Alta
CT02	Livro emprestado em 01/01	Devolver no dia 6	Devolução 07/01	Multa R\$ 0,00	Média
CT03	Livro emprestado em 01/01	Devolver com 1 dia de atraso	Devolução 09/01	Multa R\$ 2,00	Alta
CT04	Livro emprestado em 01/10	Devolver com 3 dias de atraso	Devolução 11/10	Multa R\$ 6,00	Alta
CT05	Livro emprestado em 01/01	Devolver com 24 dias de atraso	Devolução 01/02	Multa R\$ 48,00	Média
CT06	Livro emprestado em 01/01	Devolver com 25 dias de atraso	Devolução 02/02	Multa R\$ 50,00 (teto)	Alta
CT07	Livro emprestado em 01/01	Devolver quase um ano depois	Devolução 31/12	Multa R\$ 50,00 (teto)	Alta
CT08	Usuário sem pendências; 1 exemplar	Emprestar o livro L1	usuário “ana”, L1	Empréstimo registrado (1 ativo)	Alta
CT09	“ana” com 3 livros	Tentar emprestar o 4º	L4	Negado (limite de 3)	Alta
CT10	L1 com 0 exemplares livres	“bia” tenta emprestar L1	“bia”, L1	Negado (sem exemplar)	Alta
CT11	“ana” com multa de R\$ 6,00	Tentar emprestar L2	“ana”, L2	Negado (multa pendente)	Alta
CT12	“ana” com multa de R\$ 6,00	Pagar a multa e emprestar L2	“ana”, L2	Empréstimo permitido	Média
CT13	Notificador ativo	Devolver com 3 dias de atraso	“ana”, L1	Aviso enviado: (“ana”, R\$ 6,00)	Média
CT14	Notificador ativo	Devolver no prazo	“ana”, L1	Nenhum aviso enviado	Baixa

ID	Pré-condição	Passos	Dados de entrada	Resultado esperado	Prior.
CT15	“ana” com 2 livros	Emprestar L3 o 3º		Empréstimo permitido (limite não atingido)	Média
CT16	“ana” com multa pendente	“bia” (sem multa) em-presta L2	“bia”, L2	Empréstimo permitido (a multa é por usuário)	Média

Todos os casos serão automatizados em `test_biblioteca.py` (CT01-CT03 e CT05-CT06 pelo teste parametrizado `test_valores_limite_da_multa`).

4. Parte 3 - TDD e automação

Linguagem: **Python 3.14**, framework **pytest**, cobertura com **pytest-cov** (coverage.py). Código em `biblioteca.py`, testes em `test_biblioteca.py`.

4.1 Ciclos Red-Green-Refactor

Cada etapa virou um commit no repositório git (`git log --oneline --reverse`, do mais antigo ao mais novo, apenas os commits de código):

```
365969d test: red - calculo de multa por atraso
030e541 feat: green - multa de R$ 2,00 por dia de atraso
a43b60d refactor: extrai constantes de prazo e multa
9943459 test: red - teto da multa
30b13a2 feat: green - teto de R$ 50,00 na multa
2715822 refactor: nomeia dias de atraso e simplifica calculo
5f4a0ea test: red - regras de emprestimo
c278c69 feat: green - regras de emprestimo e devolucao
7c9f261 refactor: extrai constante LIMITE_LIVROS
b46a015 test: red - pagamento de multa, notificador e valores limite
3e6eb96 feat: green - pagamento de multa e notificacao
d6e7ccf refactor: separa validacao do emprestimo em metodo proprio
c34d82a test: multa de um usuario nao bloqueia outro
```

Ciclo	Red (teste que falha)	Green (código mínimo)	Refactor
1	<code>calcular_multa</code> não existia (<code>ModuleNotFoundError</code>).	<code>atraso * 2, 0</code> se não houver <code>atraso</code> .	Constantes <code>PRAZO_DIAS</code> e <code>MULTA_POR_DIA</code> no lugar de números soltos.
2	Teste do teto: obtido R\$ 714 (sem teto) e esperado R\$ 50.	<code>min(multa, 50)</code> .	Variável <code>dias_atraso</code> ; a fórmula passou a ter nomes que explicam a regra.

Ciclo	Red (teste que falha)	Green (código mínimo)	Refactor
3	5 testes de regras de empréstimo (classe Biblioteca inexistente).	Classe Biblioteca com emprestar, devolver e EmprestimoNegado.	Constante LI-MITE_LIVROS e mensagem de erro derivada dela.
4	3 testes falhando: pagar_multa inexistente e construtor sem notificador.	pagar_multa e chamada a notificador.avisar.	As 3 validações de emprestar foram para o método _validar_emprestimo.

No ciclo 4, o teste parametrizado dos valores-limite já nasceu passando, porque calcular_multa existia desde o ciclo 1. Ele entrou junto para documentar as fronteiras.

Depois do ciclo 4, foi acrescentado o teste `test_multa_de_um_usuario_nao_bloqueia_outro` (CT16), que já nasceu passando: ele cobre uma lacuna, garantindo que a multa de um usuário não bloqueia outro.

4.2 Testes automatizados

Total: **18 testes unitários** (13 funções, sendo uma parametrizada com 6 valores), cobrindo as regras RN1-RN6. Recursos exigidos:

- **Testes parametrizados:** `test_valores_limite_da_multa` com `@pytest.mark.parametrize`, 6 pares (dias → multa).
- **Mock:** `Mock()` no lugar do notificador, verificando com `assert_called_once_with("ana", 6)` e `assert_not_called()`. Isso evita depender de e-mail/SMS real.

Para rodar: `pytest -v`.

4.3 Cobertura

Comando: `pytest --cov=biblioteca --cov-branch`

Name	Stmts	Miss	Branch	BrPart	Cover
biblioteca.py	42	0	8	0	100%

Comentário: 100% das linhas e dos 8 desvios (branches) foram executados pelos testes. Isso significa que todo o código foi percorrido, mas **não** garante que o sistema esteja livre de defeitos. Cobertura mede o que foi executado, não se as verificações (asserts) são boas o bastante nem se faltam regras. Os defeitos da seção 5.2 mostram isso: o defeito do limite de livros só foi detectado porque existe um teste escrito exatamente com o valor da fronteira.

5. Parte 4 - Execução, defeitos e ferramentas

5.1 Execução dos testes

Resultado da execução completa (`pytest -v`): **18 aprovados, 0 reprovados, 0 bloqueados**.

Teste	Caso	Resultado
test_sem_atraso_nao_tem_multa	CT01	Aprovado
test_atraso_de_3_dias_gera_multa_de_6_reais	CT04	Aprovado
test_multa_nao_passa_do_teto_de_50_reais	CT07	Aprovado
test_usuario_sem_pendencias_pode_emprestar	CT08	Aprovado
test_empresta_terceiro_livro_dentro_do_limite	CT15	Aprovado
test_multa_de_um_usuario_nao_bloqueia_outro	CT16	Aprovado
test_nao_empresta_quarto_livro	CT09	Aprovado
test_nao_empresta_livro_sem_exemplar	CT10	Aprovado
test_nao_empresta_com_multa_pendente	CT11	Aprovado
test_pagar_multa_libera_novo_emprestimo	CT12	Aprovado
test_devolucao_com_multa_notifica_usuario	CT13	Aprovado
test_devolucao_no_prazo_nao_notifica	CT14	Aprovado
test_valores_limite_da_multa[6-0]	CT02	Aprovado
test_valores_limite_da_multa[7-0]	CT01	Aprovado
test_valores_limite_da_multa[8-2]	CT03	Aprovado
test_valores_limite_da_multa[31-48]	CT05	Aprovado
test_valores_limite_da_multa[32-50]	CT06	Aprovado
test_valores_limite_da_multa[33-50]	CT06	Aprovado

5.2 Defeitos introduzidos propositalmente

Os defeitos foram inseridos em uma cópia do código (o repositório continua correto) e os 18 testes foram executados contra cada cópia.

BUG-01 - Sistema permite emprestar o 4º livro

- **Defeito introduzido:** `if self.emprestimos_ativos(usuario) >= LIMITE_LIVROS` trocado por `>`.
- **Passos para reproduzir:** cadastrar 4 livros; emprestar L1, L2 e L3 para “ana”; tentar emprestar L4 para “ana”.
- **Resultado esperado:** EmpréstimoNegado (limite de 3 livros).
- **Resultado obtido:** o empréstimo é feito. O teste `test_nao_empresta_quarto_livro` falhou com `DID NOT RAISE EmpréstimoNegado` (1 reprovado, 17 aprovados).
- **Severidade:** Média (o acervo é afetado, mas não há perda de dados). **Prioridade:** Média.

BUG-02 - Multa calculada com prazo de 8 dias

- **Defeito introduzido:** `PRAZO_DIAS = 7` trocado por 8.
- **Passos para reproduzir:** emprestar um livro em 01/10/2026 e devolver em 11/10/2026.
- **Resultado esperado:** multa de R\$ 6,00 (3 dias de atraso).
- **Resultado obtido:** multa de R\$ 4,00 (`assert 4 == 6`). O teste de notificação também falhou (Expected: `avisar('ana', 6)`, Actual: `avisar('ana', 4)`). Ao todo, 5 testes reprovaram (os valores 8, 31 e 32 dias, o de 3 dias e o de notificação) e 13 aprovaram.
- **Severidade:** Alta (o usuário é cobrado a menos, perda financeira). **Prioridade:** Alta.

Ambos os defeitos foram detectados pelos testes, o que mostra o valor dos valores-limite e da regressão automatizada. O código do repositório continua passando em 18/18.

5.3 Comparação de ferramentas

Ferramenta	Foco	Pontos fortes	Pontos fracos
Cypress (ou Selenium)	Testes de interface web	Simula o clique do bibliotecário no balcão; Cypress é mais simples de configurar.	Testes mais lentos e frágeis a mudanças de tela.
Postman	Testes de API	Fácil de montar requisições (POST /emprestimos, POST /devolucoes) e validar status e JSON.	Não mede carga pesada.
JMeter	Testes de desempenho	Simula centenas de usuários ao mesmo tempo e gera gráficos de tempo de resposta.	Interface antiga e curva de aprendizado maior.

Para o módulo da biblioteca eu usaria: **Cypress** para a interface (fluxo do balcão de empréstimo), **Postman** para a API (regras RN1-RN6 via requisições) e **JMeter** para o desempenho (pico de empréstimos no início do semestre).

6. Parte 5 - Conclusão

6.1 Análise crítica

- Todas as 6 regras de negócio ficaram cobertas, com 18 testes aprovados e 100% de cobertura de linhas e branches.
- O TDD obrigou a pensar primeiro no comportamento esperado. Cada ciclo começou com um teste falhando, o que confirma que o teste realmente verifica algo antes de existir o código.
- Os 2 defeitos inseridos foram detectados, mas a cobertura de 100% sozinha não provaria isso. O que pegou os defeitos foram os testes de valor limite.

6.2 Limitações

- Os dados ficam só em memória (sem banco de dados) e não há identificação real de usuários.
- Prazo em dias corridos, sem tratar feriados e fins de semana.
- O notificador é um mock; o envio real de mensagem não foi testado.
- A multa pendente é tratada como tudo ou nada (não há pagamento parcial).
- Não foi feito teste de desempenho nem de interface, apenas descritos como planejados.

6.3 Melhorias

- Persistir dados em banco e criar testes de integração reais.
- Incluir renovação e reserva de livros, com novas regras e testes.
- Usar teste de mutação (ex.: mutmut) para medir a qualidade dos testes de forma automática, em vez de injetar defeitos manualmente.
- Receber a data atual por um relógio injetável, para facilitar testes de datas.
- Executar os testes automaticamente a cada commit (integração contínua).

7. Anexos

Repositório: <https://github.com/MvitorLS/testes-software-biblioteca> (branch feat/aproveitamento), com `biblioteca.py`, `test_biblioteca.py`, o histórico git dos ciclos TDD e este relatório.